



**Faculté
des Sciences
& Techniques**

Université de Bretagne Occidentale

Master 2 Logiciels pour Systèmes Embarqués

Simulation de mission collaborative de drones intégrant des pannes

Présenté par :
Alexandre COUTÉ

Sous la direction de :
Catherine DEZAN & Isadora FERRÃO
Université de Bretagne Occidentale | Lab-STICC

Année Universitaire 2025-2026

12 Mars 2026

Remerciements

Je tiens tout d'abord à remercier mes encadrantes, Catherine Dezan et Isadora Ferrão, pour leur accompagnement tout au long de ce projet. Ce travail m'a également offert la formidable opportunité de présenter mes recherches en anglais lors du workshop "Capes Pró Defesa" en décembre, un événement collaboratif enrichissant entre l'Université de Bretagne Occidentale (UBO) et l'Université de São Paulo (USP).

Résumé / Abstract

L'allocation de tâches multi-robots (MRTA) permet à une flotte de robots de collaborer dans un environnement pour effectuer un ensemble de tâches. Cependant, dans des conditions réalistes, ces systèmes peuvent tomber en panne et certains algorithmes, par leur rigidité ou leur centralisation, vont ainsi faire échouer des missions entières. Ce rapport présente des recherches sur les algorithmes les mieux adaptés aux cas de pannes mais aussi une étude et comparaison de trois algorithmes : l'*algorithme hongrois* (centralisé), le *Distributed Matching-by-Clone Hungarian-Based Algorithm (DMCHBA)* et le *Performance Impact (PI)*. Les trois algorithmes ont été implémentés et simulés dans un environnement sur le logiciel Coppeliasim. L'évaluation obtenue montre que les 3 algorithmes parviennent à finir la mission en cas de pannes moteur, avec une légère supériorité d'optimalité pour l'*algorithme hongrois centralisé*, mais sa centralisation reste néanmoins un défaut à prendre en compte. Ces travaux démontrent la résilience et l'efficacité des architectures décentralisées pour la gestion des ressources en environnement imprévisible.

Table des matières

Remerciements	1
Résumé / Abstract	2
Table des figures	5
1 Introduction	6
1.1 Contexte	6
1.2 Énoncé du problème	6
1.3 Contributions	6
1.4 Aperçu des résultats	7
2 Contexte Théorique	8
2.1 L'allocation de tâches multi-robots (MRTA)	8
2.1.1 Définition	8
2.1.2 Classification	9
2.2 Contraintes opérationnelles des drones	9
3 État de l'art	11
3.1 Approches centralisées contre décentralisées	11
3.2 L'algorithme Hongrois centralisé et ses limites	11
3.3 DMCHBA (Distributed Hungarian-Based Algorithm)	12
3.4 Algorithmes basés sur le consensus (CBBA et Performance Impact)	12
4 Approche et Méthodologie	14
4.1 Architecture du système proposé	14
4.1.1 CoppeliaSim	14
4.1.2 Python (sans interface graphique)	17
4.2 Modélisation des pannes	17
4.2.1 CoppeliaSim	17
4.2.2 Python (sans interface graphique)	18
5 Évaluation et Résultats	19
5.1 Scénarios de test	19
5.2 Résultats expérimentaux	20
5.2.1 CoppeliaSim	20
5.2.2 Python - Tests complets	21

6 Conclusion et Perspectives	23
6.1 Bilan des travaux	23
6.2 Perspectives d'évolution	24
A Vidéos de démonstration	25
B Algorithmes : Pseudo-code	26
Bibliographie	28

Table des figures

3.1	Exemple d'allocation finale dans la matrice de coût d'un algorithme hongrois.	12
3.2	Exemple d'allocation finale dans la matrice de coût d'un algorithme DMCHBA.	12
4.1	Environnement de simulation dans CoppeliaSim.	15
4.2	Architecture d'un robot dans CoppeliaSim.	15
4.3	Interface de surveillance.	16
4.4	Interface de simulation de déroulement de l'algorithme PI.	16
4.5	Interface de configuration.	18
5.1	Paramètres dans l'interface de configuration.	20
5.2	Résultats des scénarios avec et sans pannes.	21
5.3	Résultats du scénario pour l'algorithme hongrois.	22
5.4	Résultats du scénario pour l'algorithme DMCHBA.	22
5.5	Résultats du scénario pour l'algorithme PI.	22
6.1	Capes Pró Defesa.	24
B.1	Pseudo-code de l'algorithme hongrois.	26
B.2	Pseudo-code de l'algorithme DMCHBA.	27
B.3	Pseudo-code de l'algorithme Performance Impact.	27

Chapitre 1

Introduction

1.1 Contexte

L'allocation de tâches multi-robots ou MRTA (Multi-Robot Task Allocation) consiste à coordonner une équipe de robots pour accomplir un ensemble de tâches tout en optimisant et en minimisant le coût total. Contrairement aux comportements d'essaim qui reposent sur de la répétition, le MRTA implique une coopération intentionnelle entre les agents.

L'objectif de ce projet est de déployer des équipes de drones autonomes pour des missions critiques. Dans des conditions opérationnelles réelles, les drones font face à des contraintes : des paramètres dynamiques comme des positions géographiques variables, des ressources limitées comme l'autonomie de la batterie et des incertitudes comme les pannes mécaniques ou les pertes de communication.

1.2 Énoncé du problème

Le MRTA est un problème d'optimisation complexe. Si les algorithmes centralisés, gérés par un seul coordinateur, fonctionnent très bien dans des environnements statiques, ils se révèlent inadaptés face à l'incertitude.

Le problème majeur de la centralisation est la création d'un point de défaillance unique (Single Point of Failure) : si le planificateur central échoue, la mission entière s'arrête. D'autres algorithmes plus complexes comme le Q-learning sont trop rigides et ont tendance à échouer dès qu'un problème inattendu survient. Il est donc nécessaire de trouver des alternatives capables de réagir dynamiquement aux pannes.

1.3 Contributions

Pour répondre à cette problématique, ce travail propose une recherche sur les types d'algorithmes MRTA les mieux adaptés aux situations de pannes. La contribution principale réside dans l'étude, l'implémentation et la comparaison de trois algorithmes distincts : l'algorithme hongrois (centralisé), le DMCHBA (décentralisé amélioré) et le Performance Impact (PI).

Ces trois approches ont été implémentées dans un environnement de simulation hybride utilisant CoppeliaSim et des scripts Python, mettant en scène une flotte de trois robots mobiles. Pour tester la robustesse de ces algorithmes, des pannes moteur programmées (au démarrage, après un certain temps, ou après un certain nombre de tâches) ont

été intégrées à la simulation.

1.4 Aperçu des résultats

Les expérimentations ont permis d'évaluer le temps calculé dans le logiciel de simulation de chaque algorithme en fonction du nombre de ressources de l'environnement, avec et sans l'inclusion de pannes.

Les résultats montrent que grâce à la réitération de l'algorithme lors de la survenue d'une panne en cours de simulation, le système parvient à résoudre la mission dans tous les cas de figure. L'évaluation permet de mettre en évidence l'efficacité des algorithmes centralisés et décentralisés.

Chapitre 2

Contexte Théorique

2.1 L'allocation de tâches multi-robots (MRTA)

2.1.1 Définition

L'allocation de tâches multi-robots est un problème important dont l'objectif est de coordonner une équipe de robots mobiles pouvant comporter des robots marins, aériens, sous-marins ou encore terrestres. Les équipes peuvent être homogènes ou hétérogènes. Le but de cette coordination de robots est d'effectuer une mission demandant de compléter un ensemble de tâches en optimisant et en minimisant le coût total de l'allocation. Ce coût est défini par le temps d'exécution ou encore la distance totale parcourue par l'ensemble des robots lors de la mission. Des exemples de missions applicables à des conditions réelles pourraient être l'extinction de feux de forêt, le secours de victimes d'avalanche ou encore la récupération et livraison de colis.

Comme expliqué précédemment, l'allocation de tâches multi-robots crée une coordination entre les robots de manière à collaborer pour exécuter différentes tâches en même temps. Il est donc important de le différencier des essaims de robots (swarm) qui reposent plutôt sur des comportements de répétition de mouvements en groupe. Dans les essaims, un seul robot dirige l'ensemble des autres robots alors qu'en MRTA chaque robot possède son propre comportement que le planificateur ou l'ensemble des robots a décidé de lui attribuer.

2.1.2 Classification

Pour ajouter du contexte technique, il est aussi important de classifier les différents types d’algorithmes. Tout d’abord, nous pouvons séparer le comportement des algorithmes **centralisés et décentralisés**.

Les algorithmes centralisés allouent les tâches à l’ensemble des robots grâce à l’intervention d’un ordinateur central (ou planificateur) qui effectue l’algorithme puis communique à chaque robot individuellement leurs tâches à accomplir. Les robots n’ont donc qu’à recevoir les informations du planificateur et effectuer les tâches demandées.

Les algorithmes décentralisés ne possèdent pas de planificateur central. Ce sont les robots eux-mêmes qui exécutent l’algorithme d’allocation et discutent entre eux pour se mettre d’accord sur lequel d’entre eux doit effectuer quelles tâches.

En plus de la distinction d’exécution centralisée et décentralisée, la **taxonomie de Gerkey et Matarić** [3] permet de différencier les algorithmes par leurs capacités à effectuer certains types de missions. La taxonomie est basée sur trois axes : **les capacités du robot, les exigences pour effectuer une tâche et le temps d’affectation des tâches**.

Pour les capacités des robots, on cherche à définir si l’algorithme permet à un robot d’effectuer une seule tâche à la fois ou plusieurs tâches en même temps (**Single-Task/Multi-Task Robots (ST/MT)**).

Concernant les exigences pour effectuer les tâches, on distingue les tâches effectuelles par un seul robot des tâches effectuelles uniquement par plusieurs robots (**Single-Robot/Multi-Robot Tasks (SR/MR)**).

Enfin, le temps d’affectation des tâches est différencié entre l’assignation instantanée de l’ensemble des tâches et l’assignation durant toute la mission des tâches (**Instantaneous/Time-Extended Assignment (IA/TA)**).

2.2 Contraintes opérationnelles des drones

Pour notre sujet d’allocation de tâches pour des missions de drones comprenant des incertitudes, il est important de comprendre ce que sont ces incertitudes et comment elles peuvent survenir.

Tout d’abord, comme évoqué précédemment dans la définition du problème MRTA, certaines missions peuvent comporter une équipe de robots hétérogène. Dans ce genre d’équipe, chaque robot ne peut pas effectuer les mêmes tâches. L’allocation doit donc prendre en compte ce paramètre pour éviter les erreurs d’allocation.

Aussi, dans des conditions réalistes et critiques, de nombreux paramètres peuvent causer des pannes. Les positions géographiques, les conditions météorologiques et d’autres éléments extérieurs à l’architecture du robot sont à prendre en compte lors de la création du robot pour éviter des pannes. Des caractéristiques plus proches de l’architecture de notre robot peuvent aussi causer des pannes. Toutes les contraintes liées aux ressources de notre robot (l’autonomie de sa batterie, la limite de sa puissance de calcul, le seuil de sa charge maximale s’il doit porter du poids, etc.) peuvent causer de l’incertitude. Enfin, dans des conditions réalistes, il est très probable que des incertitudes de communication surviennent.

Tous ces paramètres sont à prendre en compte, d'abord en les anticipant pour pouvoir les esquiver au maximum, puis en utilisant des algorithmes les mieux adaptés pour réussir la mission malgré ces pannes. Nous nous concentrons sur cette dernière méthode dans ce rapport.

Chapitre 3

État de l'art

3.1 Approches centralisées contre décentralisées

Quand on parle de problème MRTA prenant en compte des pannes, la distinction entre algorithmes centralisés et décentralisés est importante.

Pour rappel, les algorithmes centralisés allouent les tâches à l'ensemble des robots grâce à l'intervention d'un ordinateur central (ou planificateur) qui effectue l'algorithme puis communique à chaque robot individuellement leurs tâches à accomplir, tandis que les algorithmes décentralisés permettent aux robots eux-mêmes d'exécuter l'algorithme d'allocation et discutent entre eux pour se mettre d'accord sur lequel d'entre eux doit effectuer quelles tâches sans planificateur central.

C'est pour cette distinction précise entre les deux types d'algorithmes que le distribué est privilégié dans la prise en compte de pannes.

En effet, la centralisation des algorithmes crée un Single-Point Of Failure (SPOF) sur le planificateur central [4]. Si ce dernier ou si la communication entre lui et les robots tombe en panne, alors toute la mission échoue et il est impossible de réparer le système pour qu'il puisse finir la mission, car le principe même de la centralisation est que les robots attendent simplement des instructions et ne savent pas exécuter l'algorithme eux-mêmes.

3.2 L'algorithme Hongrois centralisé et ses limites

L'algorithme hongrois est une méthode d'optimisation bien connue pour l'allocation. Cet algorithme permet de résoudre le problème d'allocation en 8 étapes et nécessite 4 exécutions de l'algorithme.

Bien que cet algorithme soit mathématiquement fonctionnel, simple et optimal pour des missions simples, il comporte un risque majeur en situation d'incertitude. En effet, au niveau pratique, la centralisation de cette méthode peut créer des erreurs critiques. Si l'ordinateur central qui effectue les calculs d'allocation tombe en panne, c'est l'ensemble du système qui est compromis.

	Tâche 1	Tâche 2	Tâche 3	Tâche 4	Tâche 5	Tâche 6	Tâche 7	Tâche 8	Tâche 9	Tâche 10
Robot 1	#1	#2	////	////	////	////	////	#4	////	#3
Robot 2	////	////	#2	////	////	#1	#3	////	////	////
Robot 3	////	////	////	#3	#1	////	////	////	#2	////

FIGURE 3.1 – Exemple d’allocation finale dans la matrice de coût d’un algorithme hongrois.

3.3 DMCHBA (Distributed Hungarian-Based Algorithm)

Pour pallier les limites du modèle centralisé, des approches distribuées ont été développées, comme le DMCHBA (Distributed Hungarian-Based Algorithm) étudié par Cao et al. (2024) [5].

Cette version étendue utilise un mécanisme intelligent appelé "smart-cloning". Pour que l’algorithme fonctionne de manière optimale, on ajoute des clones virtuels pour chaque robot jusqu’à ce que la matrice d’allocation soit parfaitement carrée.

Cette méthode intelligente permet d’éviter de créer une tâche virtuelle en trop lors de la phase de clonage. Grâce à cette technique, le DMCHBA permet de résoudre le problème d’allocation beaucoup plus rapidement : en seulement 5 étapes et 1 seule exécution d’algorithme.

	Tâche 1	Tâche 2	Tâche 3	Tâche 4	Tâche 5	Tâche 6	Tâche 7	Tâche 8	Tâche 9	Tâche 10
Robot 1	#1 (1)	////	////	////	////	////	////	////	////	////
Clone 1_1	////	#2 (2)	////	////	////	////	////	////	////	////
Clone 1_2	////	////	////	////	////	////	////	////	////	#3 (4)
Clone 1_3	////	////	////	////	////	////	////	#4 (8)	////	////
Robot 2	////	////	////	////	////	#1 (2)	////	////	////	////
Clone 2_1	////	////	#2 (3)	////	////	////	////	////	////	////
Clone 2_2	////	////	////	////	////	////	#3 (5)	////	////	////
Robot 3	////	////	////	////	#1 (4)	////	////	////	////	////
Clone 3_1	////	////	////	////	////	////	////	////	#2 (5)	////
Clone 3_2	////	////	////	#3 (8)	////	////	////	////	////	////

FIGURE 3.2 – Exemple d’allocation finale dans la matrice de coût d’un algorithme DMCHBA.

3.4 Algorithmes basés sur le consensus (CBBA et Performance Impact)

Outre l’algorithme hongrois, les méthodes d’allocation décentralisées s’appuient souvent sur des algorithmes de consensus, dont le plus connu est le Consensus-Based Bundle Algorithm (CBBA). Cependant, bien que le CBBA regroupe efficacement les tâches similaires, il possède une lacune importante : il ne se concentre pas sur leur performance.

C’est pour combler ce manque que l’algorithme Performance Impact (PI), développé par Zhao et al. (2016) [2], a été introduit. Le PI est une méthode distribuée basée sur le CBBA, mais qui permet à chaque robot de calculer l’impact de l’ajout d’une tâche à son planning. Le coût est défini par la performance du robot (comme sa vitesse) et sa distance par rapport aux tâches.

Chaque robot exécute l'algorithme de manière autonome à travers un cycle itératif composé de trois phases :

- **Phase 1 : Inclusion.** Le robot cherche à inclure des tâches avec le coût minimal local. Chaque robot calcule le coût (l'importance) de l'ajout d'une nouvelle tâche à son planning local et sélectionne la tâche présentant le gain marginal le plus élevé.
- **Phase 2 : Consensus.** Il s'agit d'une phase de négociation de tâches avec les voisins. Les robots communiquent avec leurs voisins, échangeant des listes de tâches et des valeurs de « pertinence » pour déterminer qui est le mieux adapté à chaque tâche. L'objectif est d'atteindre une convergence caractérisée par la stabilité (il n'y a plus de conflit en phase de consensus) et la faisabilité (chaque tâche est attribuée à un seul robot).
- **Phase 3 : Suppression.** Cette phase met à jour le système local. Si un robot se rend compte qu'un voisin peut effectuer une tâche plus efficacement (à moindre coût), il supprime cette tâche de sa propre liste.

Ce cycle est répété si le système n'a pas encore convergé. Cette prise en compte des coûts et cette négociation locale permettent au PI d'afficher une meilleure performance sur tous les points comparé au CBBA.

Chapitre 4

Approche et Méthodologie

4.1 Architecture du système proposé

4.1.1 CoppeliaSim

Les trois algorithmes ont pu être mis en place dans un environnement de simulation sur le logiciel CoppeliaSim en incluant des scripts Python dans le logiciel pour les comportements des robots et extérieurs au logiciel spécifiquement pour le comportement de planificateur central dans la version centralisée. On utilise l'API ZMQ pour communiquer avec le logiciel CoppeliaSim lors de la simulation.

L'environnement mis en place comprend toujours 3 robots représentés dans le bas de la Figure 4.1 et les ressources représentées par les petits carrés rouges. Les robots sont des robots terrestres disposant de 3 roues dont 2 motrices et d'un détecteur d'obstacles à l'avant du véhicule. Si le robot détecte un obstacle, alors il recule simplement pour essayer de l'éviter en prenant une meilleure trajectoire.

Dans la Figure 4.2, la façon dont l'architecture des robots est réalisée dans le logiciel est présentée.

Une interface extérieure au logiciel a aussi été réalisée en Python pour permettre une meilleure expérience d'utilisation. L'interface de Surveillance (Figure 4.3) permet de voir en temps réel le nombre de ressources récupérées par robot mais aussi le nombre total de ressources collectées dans la simulation. L'interface Algorithm Replay (Figure 4.4) permet quant à elle de simuler le comportement de l'algorithme Performance Impact pour permettre de comprendre le fonctionnement de ces 3 phases.

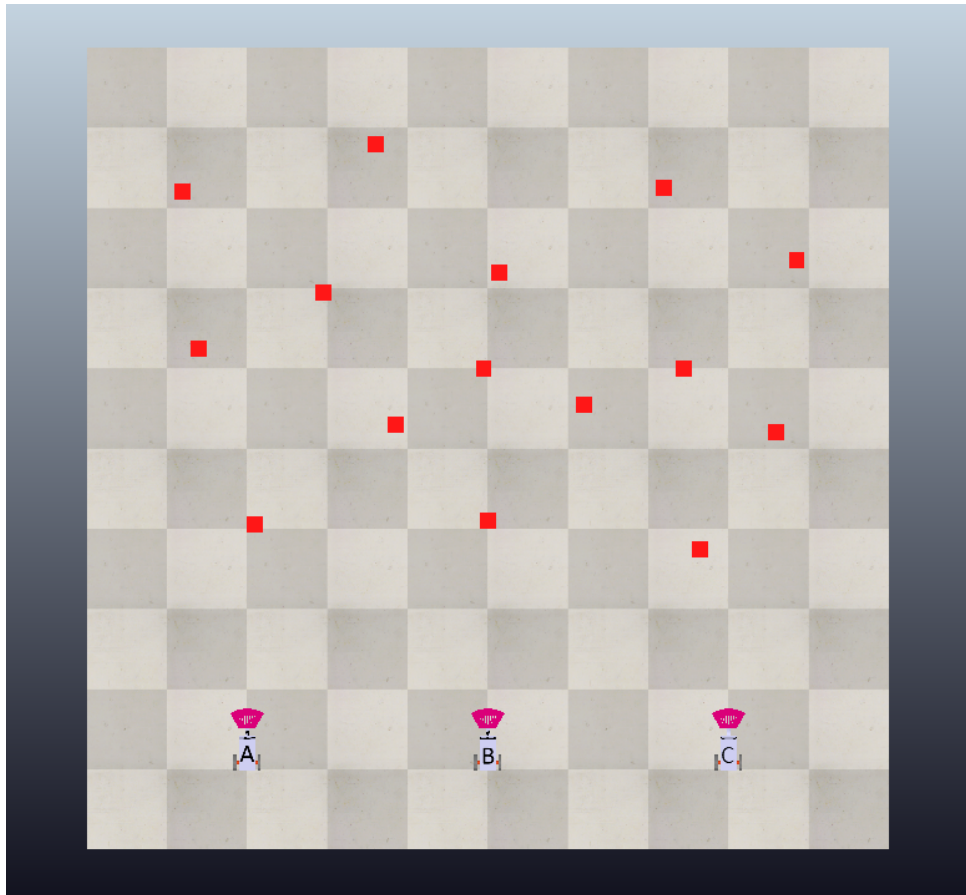


FIGURE 4.1 – Environnement de simulation dans Coppeliasim.



FIGURE 4.2 – Architecture d'un robot dans Coppeliasim.

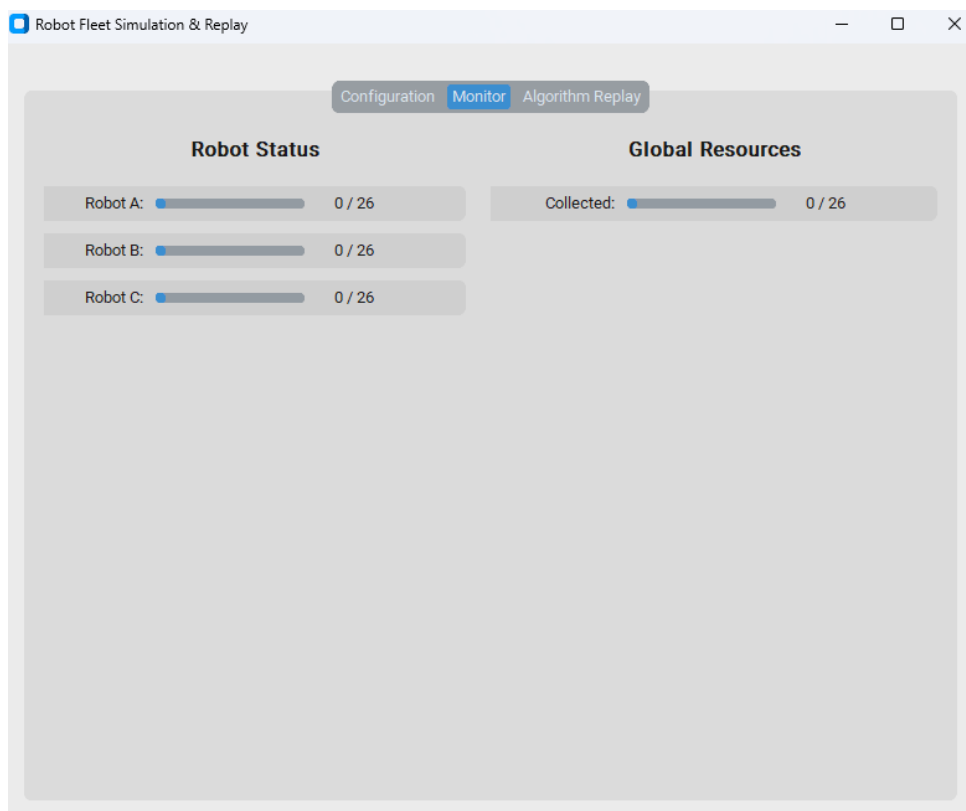


FIGURE 4.3 – Interface de surveillance.

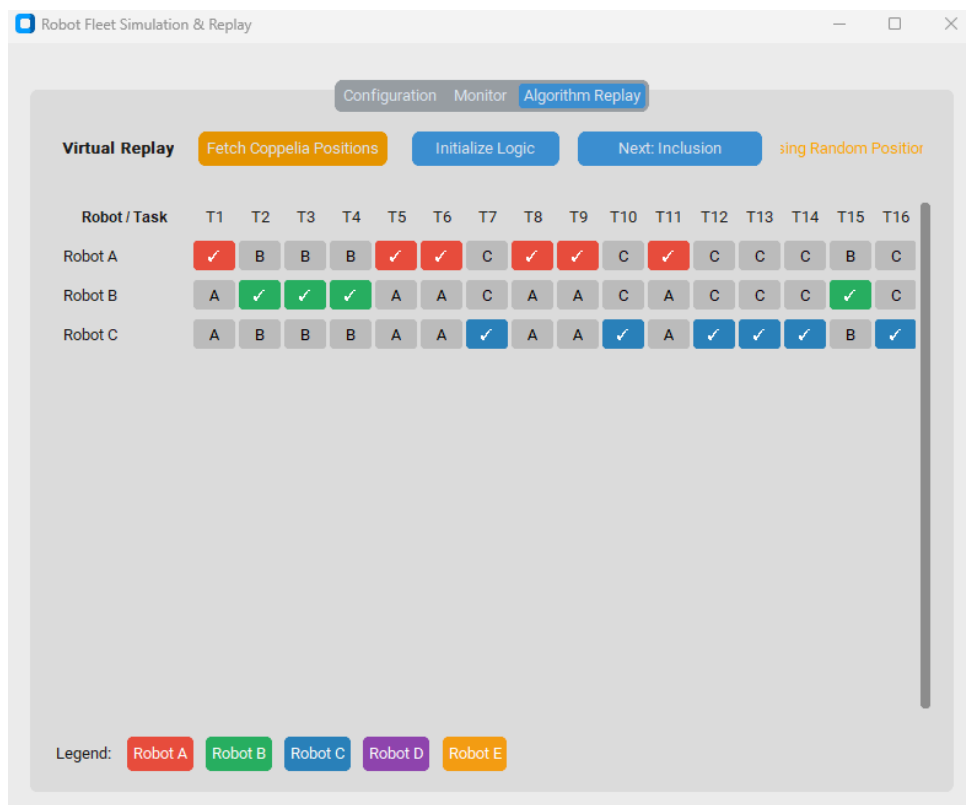


FIGURE 4.4 – Interface de simulation de déroulement de l’algorithme PI.

4.1.2 Python (sans interface graphique)

Pour accroître la pertinence des tests, les algorithmes ont été portés sur un environnement Python indépendant de CoppeliaSim. Cette version "headless" (sans interface) élimine les obstacles liés à la simulation physique. L'intégration d'un système de seed permet la reproductibilité des scénarios pour un couple ressources-robots donné. La performance finale est établie sur la moyenne de plusieurs simulations, assurant ainsi la stabilité statistique des résultats.

4.2 Modélisation des pannes

4.2.1 CoppeliaSim

Les pannes implémentées dans l'environnement de simulation sont des pannes moteur. Elles surviennent au cours de la simulation et sont individualisées pour chaque robot. La planification de la panne est configurable via l'interface Configuration (Figure 4.5). Elles peuvent survenir au Démarrage de la simulation (start), après un certain nombre de tâches (task) ou après un certain temps calculé par le logiciel de simulation (time).

Il est nécessaire de préciser que seuls les robots concernés par la panne sont au courant de leur propre panne et qu'aucune modification de leur allocation de tâches n'est effectuée due à celle-ci, ils se comportent comme s'ils n'allaient jamais tomber en panne jusqu'à leur panne.

Lors de la simulation, quand une panne survient, le robot qui tombe en panne moteur prévient les autres robots de sa panne (et l'ordinateur central en centralisé). L'algorithme est donc exécuté de nouveau avec les robots restants en remettant dans l'ensemble des tâches à allouer toutes les tâches non-effectuées.

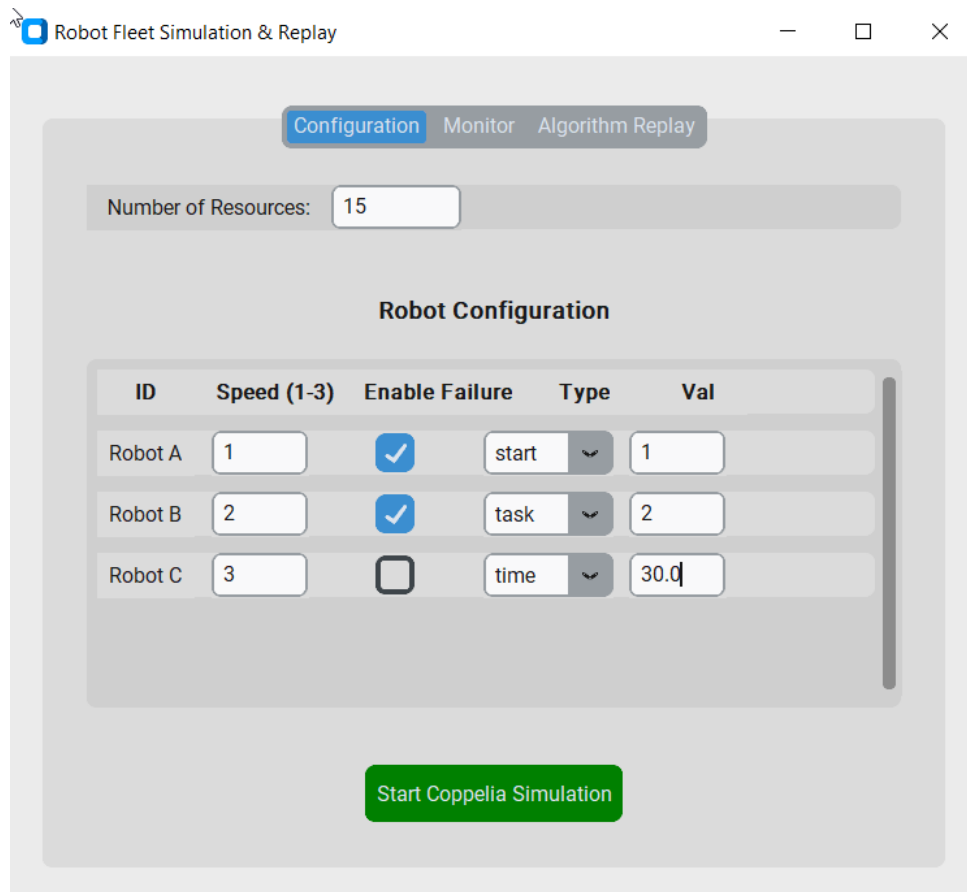


FIGURE 4.5 – Interface de configuration.

4.2.2 Python (sans interface graphique)

Dans la version Python, les pannes moteur surviennent de manière aléatoire durant le cycle de vie du robot. Afin de garantir la cohérence des tests, j'ai introduit un paramètre d'injection de pannes : un taux de 50 %, par exemple, implique que la moitié des robots de la simulation tomberont en panne durant leur mission.

Chapitre 5

Évaluation et Résultats

5.1 Scénarios de test

Pour les scénarios de test, des tests simples et rapides ont été privilégiés pour pouvoir les tester avec 3 robots avec une vitesse constante de 3 (vitesse maximale proposée) et un nombre de ressources variant de 16 à 30. Le but pour les robots est simplement de récupérer la totalité des ressources existantes dans un environnement donné.

Deux scénarios ont été mis en place et testés. Le premier est un scénario de simulation sans pannes pour les 3 algorithmes. Le second est un scénario où 2 des 3 robots sont tombés en panne moteur. Pour des résultats plus cohérents entre les algorithmes, la planification de pannes selon le nombre de tâches effectuées pour les deux robots a été privilégiée. La panne du premier robot survenant juste après sa première tâche et la panne du second robot survenant après sa seconde tâche (Figure 5.1).

Le choix de la planification des pannes après un certain nombre de tâches est dû au fait que les algorithmes possèdent des temps de calcul différents, de par leur centralisation ou décentralisation mais aussi leur complexité. Ce temps de calcul est pris en compte dans le temps compté par le logiciel de simulation. Il est donc important de privilégier que la panne survienne au même endroit pour chaque robot plutôt qu'au même temps simulé pour faire une comparaison cohérente.

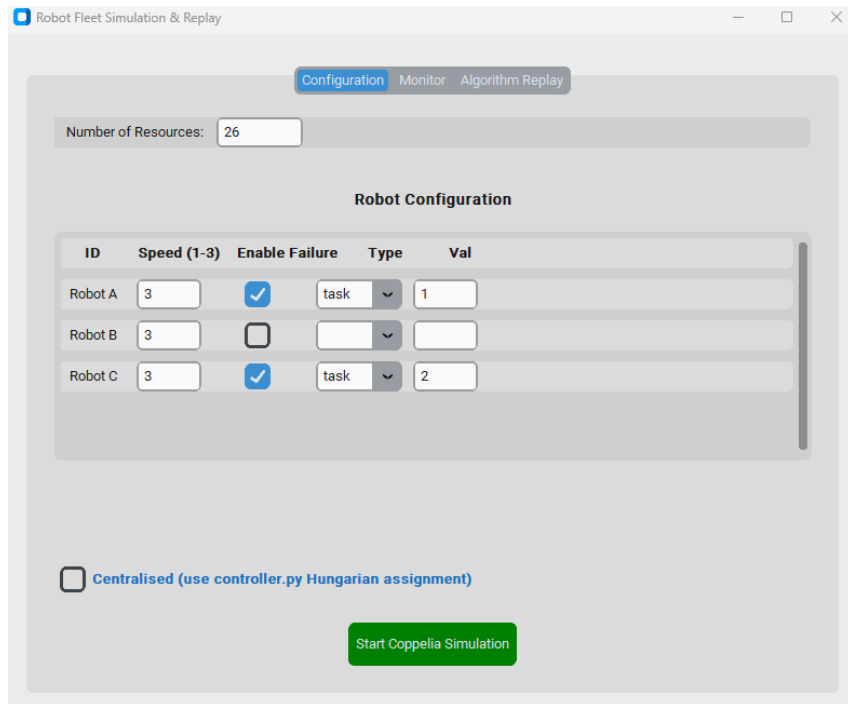


FIGURE 5.1 – Paramètres dans l’interface de configuration.

5.2 Résultats expérimentaux

5.2.1 CoppeliaSim

Les résultats démontrent que l’algorithme hongrois centralisé est légèrement plus rapide que les autres algorithmes. Ces résultats font sens car les discussions entre les robots pour se mettre d’accord sur une allocation finale dans les algorithmes distribués possèdent un temps de calcul plus long que la centralisation. Cependant, il est important de préciser de nouveau que les algorithmes centralisés possèdent un Single-Point Of Failure sur le planificateur central. Il est donc préférable de l’éviter.

Pour comparer les deux approches distribuées, le DMCHBA et le PI, le résultat n’est pas représentatif d’une certaine différence. Une distinction entre les deux algorithmes pourrait peut-être se dessiner si l’éventail de tests était plus large, mais aussi avec plus de robots et plus de ressources.

À ce sujet, il est important de prendre en compte que des tests avec plus de robots ont été essayés, mais le logiciel ayant des capacités limitées n’a pas permis d’avoir des résultats concluants.

Évaluation du temps de simulation selon les ressources :
 Comparaison des algorithmes Hongrois, DMCHBA et
 Performance Impact (3 robots, avec/sans pannes)

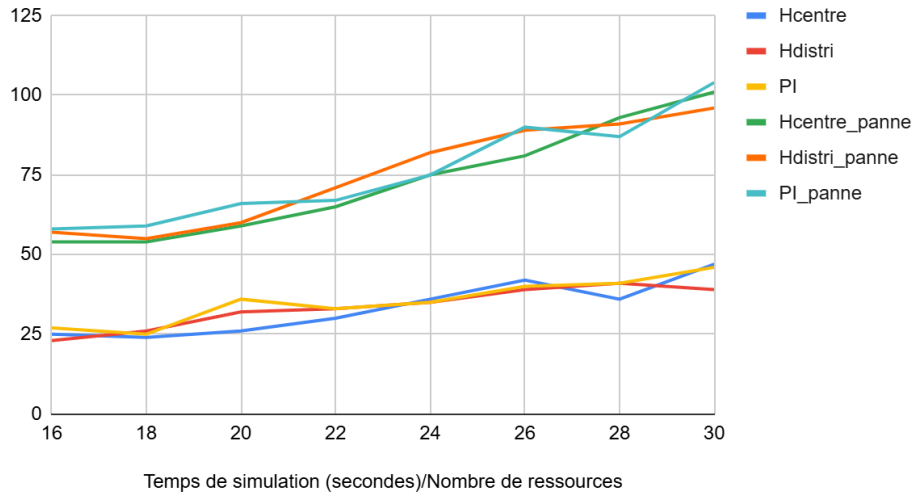


FIGURE 5.2 – Résultats des scénarios avec et sans pannes.

5.2.2 Python - Tests complets

Pour la version Python, les campagnes de tests ont été réalisées avec un nombre de robots oscillant entre 2 et 20, un volume de ressources allant de 20 à 200 et un taux de panne variant de 0 à 50 %. Les résultats, traduits sous forme de courbes via la bibliothèque Matplotlib, permettent de comparer les trois algorithmes. Bien que leurs performances globales restent similaires, les données confirment que l’algorithme DMCHBA est l’option la plus robuste en présence de pannes. En tant qu’algorithme distribué, il s’affranchit de tout point unique de défaillance (Single Point of Failure). Enfin, comparé au Performance Impact, le DMCHBA s’avère également moins exigeant en termes de puissance de calcul.

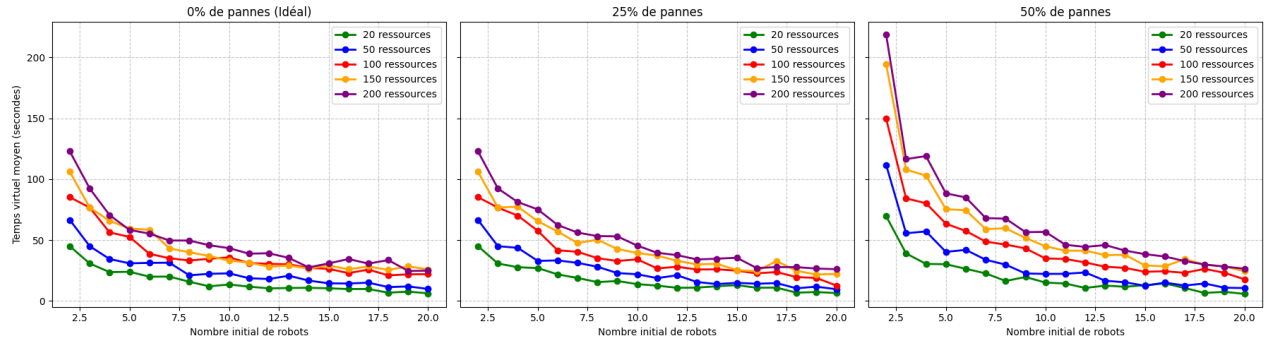


FIGURE 5.3 – Résultats du scénario pour l’algorithme hongrois.

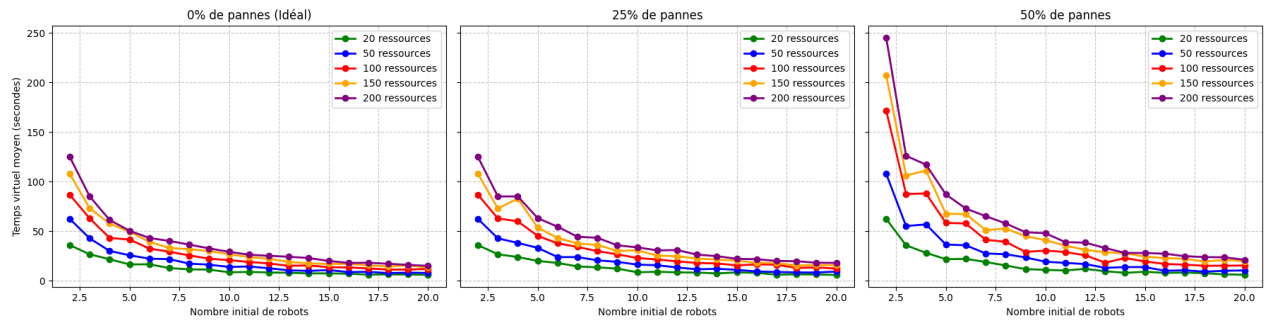


FIGURE 5.4 – Résultats du scénario pour l’algorithme DMCHBA.

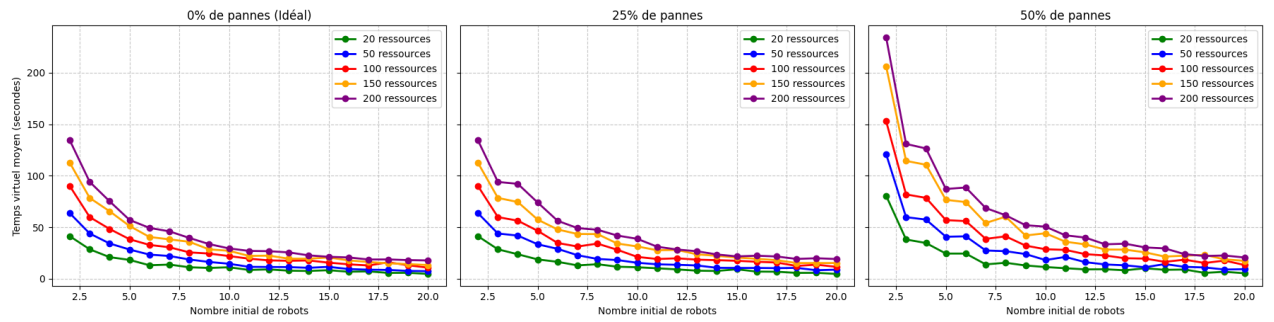


FIGURE 5.5 – Résultats du scénario pour l’algorithme PI.

Chapitre 6

Conclusion et Perspectives

6.1 Bilan des travaux

Pour conclure, ce projet a permis d'effectuer des recherches sur les types d'algorithmes d'allocation de tâches multi-robots les mieux adaptés aux pannes. Lors de ces recherches, il a été possible de se pencher sur de nombreux articles, d'état de l'art et de classification des algorithmes MRTA [1] mais aussi de nombreux articles détaillant de nombreux algorithmes comme l'algorithme des jeux hédoniques, le Consensus-Based-Bundle-Algorithm mais aussi l'Improved Performance Impact qui sont tous les trois des articles qui n'ont pas été implémentés. Cependant, ils ont pu être étudiés et comparés aux trois algorithmes mis en place avec et sans pannes moteur : l'algorithme hongrois centralisé, le DMCHBA (algorithme hongrois distribué avec smart matching-by-clone) et enfin le Performance Impact.

Ces algorithmes ont été implémentés pour des missions de récupération de ressources dans un environnement comprenant de 16 à 30 ressources et 3 robots sur le logiciel CoppeliaSim.

Une interface en Python a aussi été créée, permettant de paramétrer et de suivre en temps réel la simulation, mais aussi réalisant une démonstration technique de la phase de Consensus de l'algorithme Performance Impact.

Lors de ce projet, les encadrantes ont aussi offert l'opportunité de présenter ce travail lors du workshop "Capes Pró Defesa" le 12 décembre 2025. Ce workshop avait lieu dans le cadre d'une collaboration entre plusieurs universités, notamment l'Université de Bretagne Occidentale (UBO) et l'Université de São Paulo (USP). Il portait sur la cybersécurité et le comportement des drones.



FIGURE 6.1 – Capes Pró Defesa.

6.2 Perspectives d'évolution

Pour développer davantage ce projet, plusieurs axes sont à explorer. Des pannes plus complexes à mettre en place et intéressantes à étudier pourraient être implémentées, comme les pannes de communication par exemple. Aussi, la version améliorée du *Performance Impact (Improved PI)* pourrait être mise en place avec des missions plus complexes avec plus de robots et plus de ressources. Cependant, il faut noter que les performances du logiciel sont limitées. Il faudrait donc passer les tests sur un logiciel différent si l'on veut augmenter le nombre de robots. Des missions plus complexes pourraient aussi permettre de vérifier les limites des algorithmes. Enfin, pour pouvoir être certain du bon fonctionnement des algorithmes pour les cas de pannes, le mieux serait de réaliser des missions avec de vrais robots. Les communications entre les robots ne seraient cette fois pas simulées mais réelles, ce qui pourrait engager de nouvelles erreurs d'allocation à étudier.

Annexe A

Vidéos de démonstration

Afin d'illustrer le fonctionnement de la simulation et des algorithmes implémentés, deux vidéos de démonstration ont été réalisées et publiées sur la plateforme YouTube :

- **Performance Impact Demo with Engine Failures** : Vidéo présentant la simulation avec l'intégration de pannes moteur.

Lien : <https://youtu.be/NKEQDP3DuyE>

- **Performance Impact Demo** : Vidéo présentant le déroulement classique de l'algorithme sans panne.

Lien : <https://youtu.be/7jnT0Cy0X2E>

Annexe B

Algorithmes : Pseudo-code

Ici, la structure de chaque solution — qu'il s'agisse de l'algorithme Hongrois, du DMCHBA ou de Performance Impact — est ici décrite en pseudo-code pour en faciliter la compréhension indépendamment de l'implémentation Python :

— **Algorithme hongrois (centralisé)** :

Algorithm 1: Hungarian Algorithm	
Data: Cost matrix $C \in R^{n \times n}$	
Result: Optimal assignment minimizing total cost	
1	foreach row i in C do
2	$C[i, :] \leftarrow C[i, :] - \min(C[i, :]);$
3	foreach column j in C do
4	$C[:, j] \leftarrow C[:, j] - \min(C[:, j]);$
5	Cover all zeros in C with a minimum number of lines;
6	while number of lines $< n$ do
7	Find the minimum uncovered value m ;
8	Subtract m from all uncovered elements;
9	Add m to all elements at intersections of lines;
10	Repeat the covering step;
11	Find an optimal assignment using the zero elements (e.g., by maximum matching);

FIGURE B.1 – Pseudo-code de l'algorithme hongrois.

— DMCHBA :

Algorithm 1 Initialization

For robot i in the beginning:

- 1: **procedure** INITIALIZE ATTRIBUTES
- 2: Robot i knows its own actual attribute $A_{r_i}^*$
- 3: **for** each task j **do**
- 4: **if** robot i knows task j 's actual attribute $A_{t_j}^*$ **then**
- 5: $A_{i,j}^t(\tau_0) = A_{t_j}^*$
- 6: **else**
- 7: $A_{i,j}^t(\tau_0) = \infty$
- 8: **end if**
- 9: **end for**
- 10: **end procedure**
- 11: **procedure** INITIALIZE COMMUNICATION NETWORK
- 12: **for** each robot k **do**
- 13: **if** robot k is within robot i 's sensing range **then**
- 14: $g_{ik}(\tau_0) = 1$
- 15: **else**
- 16: $g_{ik}(\tau_0) = 0$
- 17: **end if**
- 18: **end for**
- 19: **end procedure**

Algorithm 3 Allocation Process

For robot i at time τ :

- 1: **procedure** CONSTRUCT COST MATRIX $C^i(\tau)$
- 2: **for** $\forall p \in \mathcal{R}$ **do**
- 3: **for** $\forall q \in \mathcal{T}$ **do**
- 4: **if** $A_{i,p}^r(\tau) \neq \infty$ **and** $A_{i,q}^t(\tau) \neq \infty$ **then**
- 5: $c_{j,k}^i(\tau)$ is derived using $A_{i,j}^r(\tau)$ and $A_{i,k}^t(\tau)$
- 6: **else**
- 7: $c_{j,k}^i(\tau) = \infty$
- 8: **end if**
- 9: **end for**
- 10: **end for**
- 11: **end procedure**
- 12: $[C^i(\tau), C_{\text{in}}^i(\tau)] =$
- 13: **Call Function** SQUARE_BY_CLONE($n_r, n_t, C^i(\tau), A_{r_i}^r(\tau)$)
- 14: (Only the capacity information is needed here in $A_{r_i}^r(\tau)$)
- 15: **procedure** ALLOCATION
- 16: $X^i(\tau) = \text{HUNGARIAN}(n_r, n_t, C^i(\tau))$
- 17: **end procedure**

Algorithm 2 Consensus Process

For robot i at time τ :

- 1: **procedure** COMMUNICATION
- 2: SEND $msg_i(\tau)$ to neighbor robot k with $g_{ik}(\tau) = 1$
- 3: RECEIVE $msg_k(\tau)$ from robot k with $g_{ik}(\tau) = 1$
- 4: **end procedure**
- 5: **procedure** UPDATE
- 6: UPDATE local knowledge base (robot/task attributes):
- 7: **for** $\forall p \in \mathcal{R}$ **do**
- 8: **if** $A_{k,p}^r(\tau) \neq \infty$ **then**
- 9: $A_{i,p}^r(\tau) = A_{k,p}^r(\tau)$
- 10: **end if**
- 11: **end for**
- 12: **for** $\forall q \in \mathcal{T}$ **do**
- 13: **if** $A_{k,q}^t(\tau) \neq \infty$ **then**
- 14: $A_{i,q}^t(\tau) = A_{k,q}^t(\tau)$
- 15: **end if**
- 16: **end for**
- 17: **end procedure**

Algorithm 4 Function: SQUARE_BY_CLONE ($n_r, n_t, \text{Cost Matrix}, \text{Capacity List}$)

- 1: **Step 1: Get the clone plan**
- 2: Clone_info = $\{0\} * n_r$
- 3: (Store the information of how many clones each robot has)
- 4: **while** ($n_r + \sum(\text{Clone_info}) < n_t$) **do**
- 5: **for** $\forall i \in \mathcal{R}$ **do**
- 6: **if** Clone_info[i] + 1 < Capacity List[i] **then**
- 7: Clone_info[i] += 1
- 8: **else if** Clone_info[i] + 1 == Capacity List[i] **then**
- 9: **break**
- 10: **end if**
- 11: **end for**
- 12: **end while**
- 13: **Step 2: Square Cost Matrix based on Clone_info:**
- 14: **for** $\forall i \in \mathcal{R}$ **do**
- 15: Insert Clone_info[i] identical rows below the i -th row of the input Cost Matrix, expanding it to $n_t \times n_t$
- 16: **end for**
- 17: **Return** Clone_info, Square Cost Matrix ($n_t \times n_t$)

FIGURE B.2 – Pseudo-code de l'algorithme DMCHBA.

— Performance Impact :

Algorithm 1 Consensus and Task Removal Phase Running on Vehicle v_i

- 1: Send the significance list γ_i and vehicle list β_i to vehicle v_j , where $g_{i,j}(t) = 1; j = 1, \dots, n; j \neq i$.
- 2: Receive the significance list γ_j and vehicle list β_j from vehicle v_j , where $g_{j,i}(t) = 1; j = 1, \dots, n; j \neq i$.
- 3: Perform the consensus procedure to update γ_i and β_i .
- 4: Find tasks intended to be removed $\mathbf{d}_i \leftarrow \mathbf{a}_i[\beta_i[\mathbf{a}_i] \neq v_i]$.
- 5: **while** $\max_{k=1}^{|\mathbf{d}_i|} \{\gamma_i^\diamond[\mathbf{d}_i] - \gamma_i[\mathbf{d}_i]\} > 0$ **and** $\mathbf{d}_i \neq \emptyset$ **do**
- 6: Remove the corresponding task given by $t_r^{(j)} \leftarrow \arg \max_{k=1}^{|\mathbf{d}_i|} \{\gamma_i^\diamond[\mathbf{d}_i] - \gamma_i[\mathbf{d}_i]\}$ from task list $\mathbf{a}_i$.
- 7: Remove task $t_r^{(j)}$ from $\mathbf{d}_i$.
- 8: Update time costs $c_{l,k}(\mathbf{a}_i)$ for the tasks followed after $t_r^{(j)}$ in $\mathbf{a}_i$.
- 9: Update significance $\gamma_i^\diamond$ for the rest of tasks in $\mathbf{d}_i$.
- 10: **end while**
- 11: Update vehicle list $\beta_i(\mathbf{d}_i) \leftarrow v_i$.

Algorithm 2 Task Inclusion Phase Running on Vehicle v_i

- 1: **while** $|\mathbf{a}_i| \leq m_i$ **do**
- 2: Compute the marginal significance list $\gamma_i^* \leftarrow [w_{1,i}^*, \dots, w_{m,i}^*]^T$.
- 3: **if** $\max_{k=1}^m \{\gamma_{i,k} - \gamma_{i,k}^*\} > 0$ **then**
- 4: $t_\ell^{(j)} \leftarrow \arg \max_{k=1}^m \{\gamma_{i,k} - \gamma_{i,k}^*\}$.
- 5: $t_\ell^{(j)} \leftarrow \arg w_{\ell,i}^*(\mathbf{a}_i \oplus t_\ell^{(j)})$.
- 6: Add task $t_\ell^{(j)}$ into $\mathbf{a}_i$ at position $t_\ell^{(j)}$.
- 7: Update significance $\gamma_{i,\ell} \leftarrow \gamma_{i,\ell}^*$.
- 8: Update vehicle list's entry $\beta_{i,\ell} \leftarrow v_i$.
- 9: Update time cost $c_{l,k}(\mathbf{a}_i)$ for the tasks followed after $t_\ell^{(j)}$ in $\mathbf{a}_i$.
- 10: **else**
- 11: **break**.
- 12: **end if**
- 13: **end while**
- 14: Update significance list $\gamma_i \leftarrow [\gamma_{i,1}, \dots, \gamma_{i,m}]^T$.

FIGURE B.3 – Pseudo-code de l'algorithme Performance Impact.

Bibliographie

- [1] Nguyen, T. T., et al. (2025). *Task Allocation for Autonomous Machines using Computational Intelligence and Deep Reinforcement Learning*. arXiv preprint.
- [2] Zhao, W., Meng, Q., & Chung, P. W. H. (2016). *A Heuristic Distributed Task Allocation Method for Multivehicle Multitask Problems and Its Application to Search and Rescue Scenario*. IEEE Transactions on Cybernetics.
- [3] Chakraa, H., et al. (2023). *Optimization techniques for Multi-Robot Task Allocation problems : Review on the state-of-the-art*. Robotics and Autonomous Systems.
- [4] Wang, Q., et al. (2023). *Multi-UAV Task Allocation Method Considering Single Point of Failure*. IEEE.
- [5] Cao, X., Liu, K., & Sun, G. (2024). *A Distributed Hungarian-Based Algorithm for Multi-Robot Task Allocation with Load Balancing*. IEEE.